



Presentation Title on openSUSE Asia Summit 2026 in Two Lines (Dark)

Speaker Name

Yogyakarta | October, 2026

An introduction to RPM's and openSUSE's tools

Repositories



- Distro's - Where new distributions are worked on
 - openSUSE:Factory
 - slfo-1.3 (Leap 16.1 git)
 - slfo-main (Next future Leap release git)
- Devel - Where multiple people can develop individual packages
 - devel:languages:python
 - Base:System
 - or git for select projects
- Home - Where individual Contributors can work on whatever they'd like
 - home:simotek

Spec Files - Example



- <https://build.opensuse.org/package/show/devel:languages:python/python-bytecode>
- <https://build.opensuse.org/package/show/devel:languages:python/python-autobahn>





Packaging for openSUSE

Simon Lees

✉ sflees@suse.de / simon@simotek.net

[Simotek](#) on [irc.libera.net](#) / [Discord](#)

🐦 [@Simotek_Dot_Net](#)

Aim



- This is a quick intro to packaging
- Just enough to get you started and know where to look
- Detailed Written Guide
<https://www.mac-vicar.eu/tutorials/rpm-packaging/index.html>
- Basic obs guide
<https://openbuildservice.org/help/manuals/obs-user-guide/art.obs.bg>
- Git workflow guide
https://en.opensuse.org/openSUSE:Git_Packaging_Workflow

What is Packaging?

1. Build and Map Files

- Compile / build source files into binaries
- Ensure other files end up in the right location.



2. Dependencies

- We want to make sure that everything the package needs to build is installed on the system.



3. Anything Else

- Create Users
- Run one time scripts
- Reload caches

Step 1: Understand your Package

Understanding



- You don't need to be a programmer to get started packaging
- Which Language / Languages are in use?
- Do you need to compile anything?
- Which Buildsystem does it use?
- How does Upstream normally deploy the package?
- Can you install the software on your machine from just the sources?

Step 2: Try and find a similar Package

Autogenerate

- For some languages such as python there are tools to autogenerate a spec file
- You can also create a empty spec file with tools like “vim foo.spec”
- Your LLM of choice may generate a reasonable starting point

Branch or Create



- If we are working on a package that already exists we can branch it into our home project
- Otherwise we need to create it.

Finding The right version



- Browse to openSUSE:Factory
 - <https://build.opensuse.org/project/show/openSUSE:Factory>
- Use the filter to find the package you want
- Select the package
- At the top right look for **Developed at XXX**
- This will give you the devel project.
- Click on it
- Select **Branch Package** on the Left.

Finding The right version (git)



- Search in
 - <https://src.opensuse.org/explore/repos>
- Use the project / package version not the pool/package version
- We recommend using **osc fork** rather than the Web UI to fork the project

Creating a Package



- Browse to your home project (not mine)
 - Shortcut on Left Bar
 - <https://build.opensuse.org/project/show/home:simotek>
- Using osc
 - osc co project
 - cd project
 - osc mkpac name
 - cd name

Step 3: Building

Files



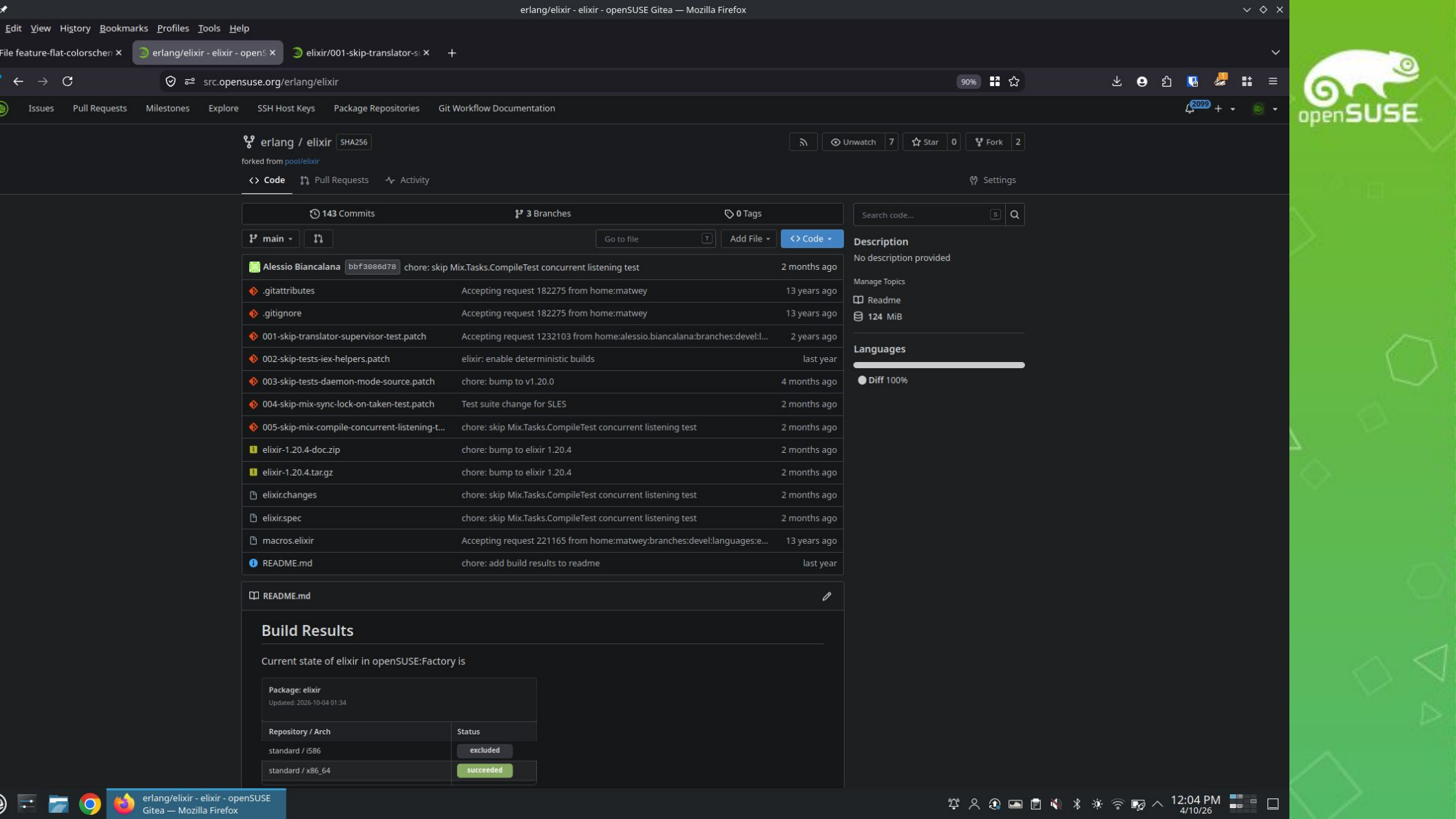
- Source code
- .spec file - used for building
- .changes file - changelog
- patches - any further changes we need to make to the sources
- Distro specific configuration files

Files - Example



- <https://build.opensuse.org/package/show/X11:Enlightenment:Factory/terminology>
- <https://src.opensuse.org/erlang/elixir>





elixir/001-skip-translator-supervisor-test.patch at main · elixir - opensUSE Gitea — Mozilla Firefox

File Edit View History Bookmarks Profiles Tools Help

File feature-flat-colorschen x elixir/001-skip-translator-s x elixir/001-skip-translator-s x +

src.opensuse.org/erlang/elixir/src/branch/main/001-skip-translator-supervisor-test.patch 120% ☆

Issues Pull Requests Milestones Explore SSH Host Keys Package Repositories Git Workflow Documentation 2099 +

erlang / elixir SHA256

forked from pool/elixir

<> Code Pull Requests Activity Settings

Files

main elixir / 001-skip-translator-supervisor-test.patch Go to file Add File

Fabrizio Sestito 352ce321f1 Accepting request 1232103 from home:alessio.biancalana:branches:devel:languages:erlang 2 years ago

23 lines | 714 B | Diff

```
1 diff --git a/lib/logger/test/logger/translator_test.exs b/lib/logger/test/logger/translator_test.exs
2 index e15010a29..cc99f945b 100644
3 --- a/lib/logger/test/logger/translator_test.exs
4 +++ b/lib/logger/test/logger/translator_test.exs
5 @@ -945,6 +945,7 @@ test "translates :proc_lib crashes with neighbour with name" do
6     ""
7     end
8
9 + @tag :skip
10     test "translates Supervisor progress" do
11         {:ok, pid} = Supervisor.start_link([], strategy: :one_for_one)
12
13     @@ -960,6 +961,7 @@ test "translates Supervisor progress" do
14         ""
15         end
16
17 + @tag :skip
18     test "translates Supervisor progress with name" do
19         {:ok, pid} = Supervisor.start_link([], strategy: :one_for_one, name: __MODULE__)
20
21
22
```



Spec Files



- Header
- %prep
- %build
- %install
- %pre / %post / %postrans / %postun
- %files

Spec Files - Header

- Name
- Version
- Sources / Patches
- Dependencies





Spec Files - %prep

- Extract sources
- Apply Patches



Spec Files - %build



- Compile binaries
 - Make
 - ./configure
 - cmake; make
 - meson; ninja
 - language specific build / macros

Spec Files - %install

- Move all the files to where they will end up.



Build Service

AS

Your Requests

Your Home Project

All Projects

Status Monitor

Collapse sidebar

terminology.spec of P: x

→ ↺ build.opensuse.org/projects/X11:Enlightenment:Factory/packages/terminology/files/terminology.spec?expand=1

130%

🔍

👁️

🔔 42364

🌱

Search (Press '/')🔍

👁️

🔔 42364

🌱

Good morning! Today we bring you three new features that will make a moderators life easier. Check them out! <https://openbuildservice.org/2026/10/01/content-moderation-becomes-stronger/>

Got it

135

136

137

138

139

140

141

142

143

144

145

146

147

148

149

150

151

152

153

154

155

156

157

158

159

160

161

162

163

164

165

166

%if 0%{?suse_version} < 1320

%post

%icon_theme_cache_post

%desktop_database_post

%postun

%icon_theme_cache_postun

%desktop_database_postun

%endif

%{?suse_version:%files}

%{!suse_version:%files -f %{name}.lang}

%defattr(-,root,root)

%doc AUTHORS COPYING ChangeLog NEWS README*

%{_bindir}/%{name}

%{_bindir}/ty*

%{_datadir}/%{name}/

%{_datadir}/applications/%{name}.desktop

%{_datadir}/icons/hicolor/*/apps/%{name}.??g

%{_mandir}/man?/terminology*.??z

%{_mandir}/man?/ty*.??z

%exclude %{_datadir}/%{name}/themes/*

%exclude %{_datadir}/%{name}/colorschemes/*

%files theme-upstream

%{_datadir}/%{name}/themes/default.edj

%{_datadir}/%{name}/colorschemes/Default.eet

%files theme-Flat

%{_datadir}/%{name}/themes/Flat.edj

%{_datadir}/%{name}/colorschemes/Flat.eet

File terminology.spec of Package terminology - openSUSE Build S...

terminology - openSUSE Build S...

🔔 👤 📄 🗑️ 🔊 🔌 🌐 🏠

1:11 PM 4/10/26

🖱️

🦎

openSUSE

Examples

Files - Example



- <https://build.opensuse.org/package/show/X11:Enlightenment:Factory/terminology>
- <https://src.opensuse.org/erlang/elixir>

Step 3: Building

Working Locally

- First we need to checkout the package
 - `osc co project/package`
- Then we can work with the files locally
- To add / remove files from the project
 - `osc ar` (Similar to git add)
- Commit changes when done
 - `osc commit` Similar to git commit + git push

Working Locally (git)

- First we need to checkout the package
 - osc fork project package
- Then we can work with the files locally
 - use git add, commit, push as normal
 - Then create a PR/MR

Building



- First configure repositories.
 - <https://build.opensuse.org/repositories/home:simotek>
- `osc build repo`
- The buildservice will automatically build any enabled packages.

Step 4: Contributing

Contributing

- Send from “home” project to “devel” project
- “devel” project maintainer will send it to Tumbleweed
- You can then send it to the next Leap
- This can be done either with a **submit request** from the webui or on the command line with **osc sr**

Contributing (git)

- Send from “home” project to “devel” project, use a PR/MR
- “devel” project maintainer will send it to Tumbleweed
- You can then send it to the next Leap
 - Create done either with a **submit request** from the webui or on the command line with **osc sr**

Files - Example



- <https://build.opensuse.org/package/show/X11:Enlightenment:Factory/terminology>
- <https://src.opensuse.org/erlang/elixir>

Staging Dashboard

- https://build.opensuse.org/staging_workflows/openSUSE:Factory



Maintenance



- Sometimes we need to change a package in an existing Leap distribution. This is now Also done via git with
- `osc fork` and `creating a pull request` when done
 - pool - slfo-1.2 (Leap 16.0)
 - pool - slfo-1.3 (Leap 16.1)
 - pool - slfo-main (The next Leap)
- If the package is used by SUSE for SLES the SUSE Maintainer may need to submit the update.

Some Git Magic

scmsync



- obs metadata to build packages from src.o.o
- now offered on the create package section
- <https://build.opensuse.org/projects/home:simotek:git-builds/packages/erlang/meta>

2 Minutes of Advanced Concepts

AI



- <https://github.com/openSUSE/openSUSE-packaging-skill>

Constraints



- Specifies minimum requirements for a builder
- `_constraints` file
- https://openbuildservice.org/help/manuals/obs-user-guide/cha.obs.build_job_constraints.html
- Example
 - https://build.opensuse.org/package/view_file/LibreOffice:Factory/libreoffice/_constraints?expand=1

rpmlintrc



- RpmLint does a good job of finding issues in packages sometimes it creates false positives though
- <packagename>-rpmlintrc file
- https://en.opensuse.org/openSUSE:Packaging_checks
- Example:
<https://build.opensuse.org/package/show/Base:System/python-dbus-python>

Supplements



- You have a vim plugin for Rails, or a
- Squid plugin for yast
 - Supplements: (A and A-Client)
 - Supplements: (vim and ruby-common-rails)

Multibuild



- Build multiple packages from the same spec and source
- `_multibuild` file
- <https://openbuildservice.org/help/manuals/obs-user-guide/cha.obs.multibuild.html>
- Example:
<https://build.opensuse.org/package/show/development:tools:building/cmake>

Project Config

- Used to define settings between different Distributions along with some rpm macro's



Language files

- Header
 - %lang_package
- Install
 - %find_lang %{name}
- Files
 - %{?suse_version:%files lang -f %{name}.lang}
- Example
 - https://build.opensuse.org/package/view_file/X11:Enlightenment:Factory/terminology/terminology.spec?expand=1

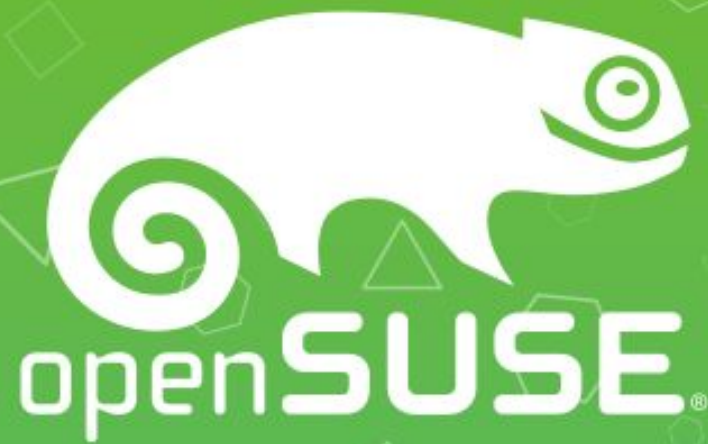
Spec Cleaner

- Improve / Update your spec files
- <https://github.com/rpm-software-management/spec-cleaner>

Questions



- <https://github.com/simotek/Talks/blob/master/Intro%20to%20Packaging%20Asia%20Summit%2026.pdf>
- Detailed Written Guide
<https://www.mac-vicar.eu/tutorials/rpm-packaging/index.html>
- Basic obs guide
<https://openbuildservice.org/help/manuals/obs-user-guide/art.obs.bg>
- Git workflow guide
https://en.opensuse.org/openSUSE:Git_Packaging_Workflow





Packaging Workshop

Simon Lees
Simotek

✉ sflees@suse.de / simon@simotek.net

Simotek on irc.libera.net / Discord

🐦 @Simotek_Dot_Net

A Hackweek Experiment

[https://en.opensuse.org/openSUSE:ALP/
Workgroups/GrassyKnoll](https://en.opensuse.org/openSUSE:ALP/Workgroups/GrassyKnoll)

Getting Started



- Create an account at <https://idp-portal.suse.com/univention/self-service/#page=createaccount> if you don't have a bugzilla / obs / openSUSE Community account.
- Install `patterns-devel-osc-devel_osc_build` (This should work with a WSL Image as well)
- `osc co home:<your-user>`

What are we doing?

- Has anyone done packaging before?
- Has anyone got any questions or something they'd like to look at?
- We can also look at building Images if you'd like.



Breaking a Package

- Add a file that doesn't exist
- Remove one that does
- Make a typo in a macro name



